

Análise de Rede via CLI (TShark/Capinfos)

1. Objetivo

Utilizar ferramentas de linha de comando (CLI) para analisar arquivos de captura de tráfego (.pcapng) de forma rápida, eficiente e automatizada, sem depender de interfaces gráficas.

2. Ferramentas

- **Capinfos:** Exibe metadados e estatísticas gerais do arquivo de captura (duração, hashes, total de pacotes).
- **TShark:** Versão CLI do Wireshark para análise profunda de pacotes e filtragem.

3. Comandos Principais

```
# Obter resumo da captura
capinfos demo.pcapng
```

```
# Filtrar tráfego HTTP em tempo real
tshark -r demo.pcapng -Y "http"
```

4. Conclusão

O uso da CLI é fundamental para analistas de SOC, permitindo a **análise forense de rede** eficiente, automação via scripts e manipulação de grandes volumes de dados (piping).

Investigação de Redes: Interface e Captura com TShark

1. Cenário/Problema

O analista de segurança precisa identificar as interfaces de rede ativas em um host e realizar a captura de pacotes em tempo real (*sniffing*) para diagnosticar comunicações suspeitas ou validar o fluxo de tráfego em portas específicas. O uso da CLI é preferível para evitar o consumo de recursos de uma GUI e permitir a filtragem rápida de grandes volumes de dados.

2. Ferramentas Utilizadas

- **TShark** (Wireshark Network Analyzer CLI).
- **Linux Terminal** (Ambiente Ubuntu).
- **Privilégios de Superusuário** (Necessários para acesso direto ao hardware de rede).

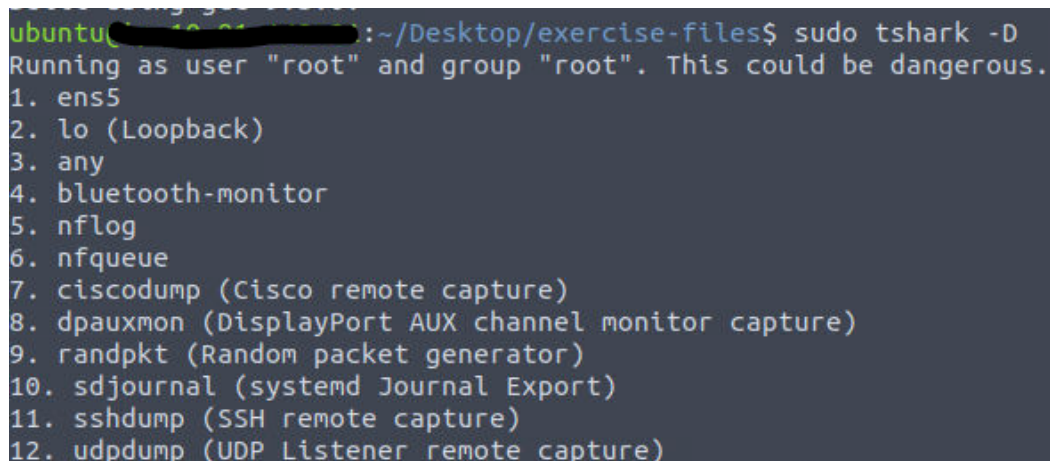
3. Evidências Técnicas (Logs/Comandos)

3.1. Identificação de Alvos de Captura

O primeiro passo técnico consiste em listar todas as interfaces onde o TShark pode "escutar". Isso é crucial em servidores com múltiplas placas de rede (NICs) ou interfaces virtuais (Docker, VPNs).

Bash

```
# Listagem de interfaces disponíveis com privilégios de root  
sudo tshark -D
```



```
ubuntu@ubuntu:~/Desktop/exercise-files$ sudo tshark -D  
Running as user "root" and group "root". This could be dangerous.  
1. ens5  
2. lo (Loopback)  
3. any  
4. bluetooth-monitor  
5. nflog  
6. nfqueue  
7. ciscodump (Cisco remote capture)  
8. dpauxmon (DisplayPort AUX channel monitor capture)  
9. randpkt (Random packet generator)  
10. sdjournal (systemd Journal Export)  
11. sshdump (SSH remote capture)  
12. udpdump (UDP Listener remote capture)
```

Legenda: Figura 1 - Saída do comando -D detalhando os índices das interfaces (ex: 1. ens5, 2. lo) prontos para monitoramento.

3.2. Execução de Sniffing em Tempo Real

A captura de pacotes inicial permite validar se o tráfego esperado está atingindo a interface correta. O TShark, por padrão, utiliza a primeira interface disponível se nenhuma for especificada.

Bash

```
# Captura de tráfego na interface principal (ens5)  
tshark
```

```
# Captura direcionada à interface de Loopback (tráfego interno/local)  
tshark -i 2
```

```

1000 packets captured
ubuntu@cp-10-81-142-21:~/Desktop/exercise-files$ tshark
Capturing on 'ens5'
  1 0.000000000 10.81.142.21 ? 10.81.73.35  TCP 4373 80 ? 55714 [PSH, ACK] Seq=1 Ack=1 Win=442 Len=4307 TSval=2826930868 TSecr=1369714747
  2 0.000233810 10.81.73.35 ? 10.81.142.21  TCP 66 55714 ? 80 [ACK] Seq=1 Ack=4308 Win=4964 Len=0 TSval=1369714836 TSecr=2826930868
  3 0.019186402 10.81.142.21 ? 10.81.73.35  TCP 9015 80 ? 55714 [ACK] Seq=4308 Ack=1 Win=442 Len=8949 TSval=2826930887 TSecr=1369714836
  4 0.019194152 10.81.142.21 ? 10.81.73.35  TCP 9015 80 ? 55714 [PSH, ACK] Seq=13257 Ack=1 Win=442 Len=8949 TSval=2826930887 TSecr=1369714836
  5 0.019205812 10.81.142.21 ? 10.81.73.35  TCP 9015 80 ? 55714 [ACK] Seq=2220 Ack=1 Win=442 Len=8949 TSval=2826930887 TSecr=1369714836
  6 0.019208402 10.81.142.21 ? 10.81.73.35  TCP 9015 80 ? 55714 [PSH, ACK] Seq=31155 Ack=1 Win=442 Len=8949 TSval=2826930887 TSecr=1369714836
  7 0.019264492 10.81.142.21 ? 10.81.73.35  TCP 9015 80 ? 55714 [ACK] Seq=4010 Ack=1 Win=442 Len=8949 TSval=2826930887 TSecr=1369714836

```

Legenda: Figura 2 - Demonstração de captura ativa mostrando o fluxo de pacotes TCP (SYN, SYN-ACK, ACK) na interface selecionada.

3.3. Parâmetros de Controle e Versão

Documentação da versão utilizada para garantir a compatibilidade de filtros de exibição e suporte a protocolos.

Bash

Verificação de versão e bibliotecas de suporte (Libpcap/OpenSSL)

tshark -v

```

ubuntu@cp-10-81-142-21:~/Desktop/exercise-files$ tshark -v
TShark (Wireshark) 3.2.3 (Git v3.2.3 packaged as 3.2.3-1)

Copyright 1998-2020 Gerald Combs <gerald@wireshark.org> and contributors.
License GPLv2+: GNU GPL version 2 or later <https://www.gnu.org/licenses/gpl-2.0.html>
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```

Legenda: Figura 3 - Detalhes da versão do TShark e permissões de licença.

4. Mapeamento MITRE ATT&CK

- **Tática:** TA0007 - Discovery
- **Técnica:** T1046 - Network Service Discovery (Utilizado para identificar serviços ativos e portas abertas via análise de tráfego).
- **Técnica:** T1040 - Network Sniffing (Ferramenta utilizada para monitorar comunicações de rede).

5. Conclusão e Mitigação

A capacidade de operar o TShark via linha de comando permite uma resposta rápida em incidentes onde o acesso é limitado ao terminal (SSH). Como medida de segurança, o acesso à execução de ferramentas de sniffing deve ser restrito a usuários com privilégios de administrador, pois a captura indevida de tráfego pode levar à exposição de credenciais em protocolos não criptografados (como HTTP ou FTP).

Investigação Avançada com TShark:

Parâmetros de Leitura, Escrita e Verbosidade

1. Cenário/Problema

Após a captura inicial, um analista de segurança precisa processar arquivos de captura (.pcapng) existentes para isolar fluxos suspeitos, exportar subconjuntos de dados para análise forense detalhada e examinar a estrutura interna dos pacotes em busca de anomalias que não são visíveis na visualização simplificada.

2. Ferramentas Utilizadas

- **TShark** (Wireshark CLI).
- **Linux Terminal**.
- **Arquivos de Captura:** demo.pcapng e write-demo.pcap.

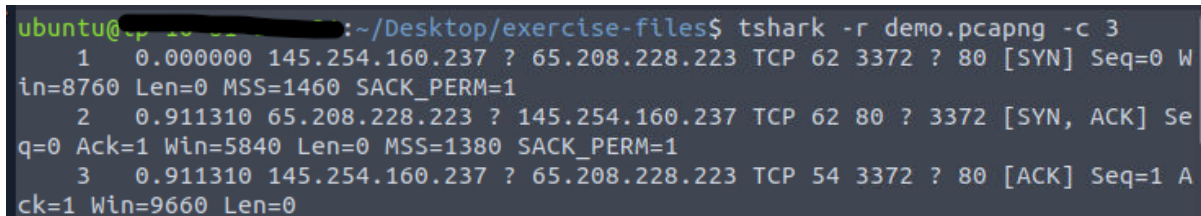
3. Evidências Técnicas (Logs/Comandos)

3.1. Processamento de Arquivos e Controle de Fluxo

A leitura de arquivos permite a análise retrospectiva. O uso do parâmetro -c é essencial para limitar a saída e evitar o transbordamento de dados no terminal.

Bash

```
# Ler o arquivo de captura e limitar a exibição aos 3 primeiros pacotes  
tshark -r demo.pcapng -c 3
```



```
ubuntu@...:~/Desktop/exercise-files$ tshark -r demo.pcapng -c 3  
  1   0.000000 145.254.160.237 ? 65.208.228.223 TCP 62 3372 ? 80 [SYN] Seq=0 W  
in=8760 Len=0 MSS=1460 SACK_PERM=1  
  2   0.911310 65.208.228.223 ? 145.254.160.237 TCP 62 80 ? 3372 [SYN, ACK] Se  
q=0 Ack=1 Win=5840 Len=0 MSS=1380 SACK_PERM=1  
  3   0.911310 145.254.160.237 ? 65.208.228.223 TCP 54 3372 ? 80 [ACK] Seq=1 A  
ck=1 Win=9660 Len=0
```

Legenda: Figura 1 - Leitura controlada de arquivo PCAP exibindo apenas o cabeçalho das comunicações iniciais.

3.2. Extração e Escrita de Dados Seleccionados

O TShark permite salvar filtros específicos em novos arquivos, facilitando o compartilhamento de evidências ou o foco em incidentes isolados.

Bash

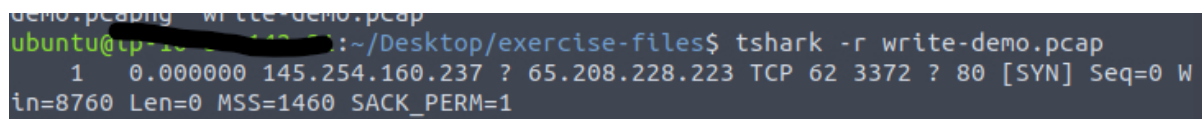
Extrair o primeiro pacote de um arquivo e salvar em um novo arquivo .pcap

```
tshark -r demo.pcapng -c 1 -w write-demo.pcap
```

Validar a criação e conteúdo do novo arquivo

ls

```
tshark -r write-demo.pcap
```



```
ubuntu@ip-10-10-10-10:~/Desktop/exercise-files$ tshark -r write-demo.pcap
1 0.000000 145.254.160.237 ? 65.208.228.223 TCP 62 3372 ? 80 [SYN] Seq=0 W
in=8760 Len=0 MSS=1460 SACK_PERM=1
```

Legenda: Figura 2 - Fluxo de trabalho de exportação de pacotes específicos para investigação forense isolada.

3.3. Análise Profunda: Verbosidade e Dump Hexadecimal

Para inspeção de payloads e cabeçalhos de protocolos, utilizamos o modo verboso (-V) e o dump em hexadecimal/ASCII (-x).

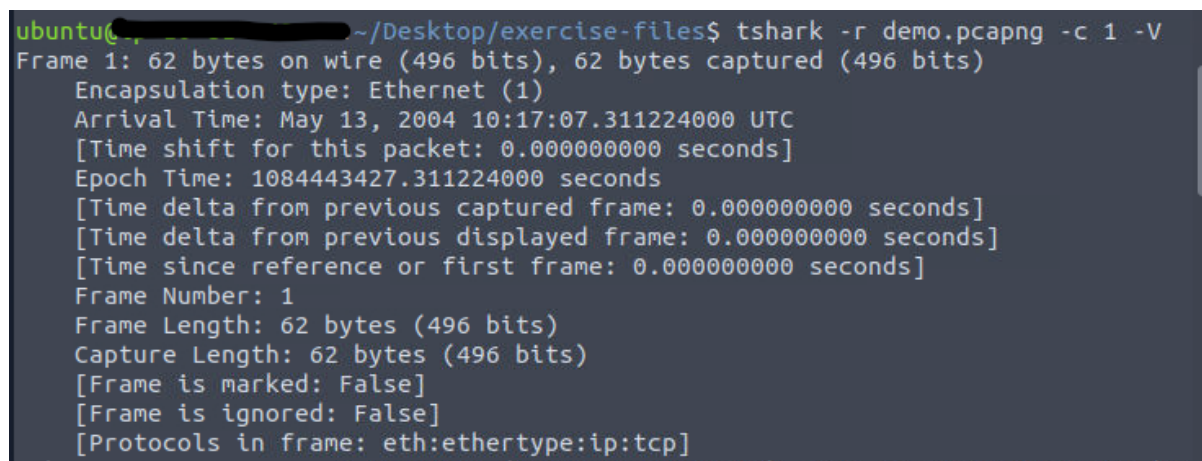
Bash

Exibir o conteúdo do pacote em formato Hexadecimal e ASCII

```
tshark -r write-demo.pcap -x
```

Exibir detalhes completos de todas as camadas do modelo OSI (Verbosidade)

```
tshark -r demo.pcapng -c 1 -V
```



```
ubuntu@ip-10-10-10-10:~/Desktop/exercise-files$ tshark -r demo.pcapng -c 1 -V
Frame 1: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)
  Encapsulation type: Ethernet (1)
  Arrival Time: May 13, 2004 10:17:07.311224000 UTC
  [Time shift for this packet: 0.000000000 seconds]
  Epoch Time: 1084443427.311224000 seconds
  [Time delta from previous captured frame: 0.000000000 seconds]
  [Time delta from previous displayed frame: 0.000000000 seconds]
  [Time since reference or first frame: 0.000000000 seconds]
  Frame Number: 1
  Frame Length: 62 bytes (496 bits)
  Capture Length: 62 bytes (496 bits)
  [Frame is marked: False]
  [Frame is ignored: False]
  [Protocols in frame: eth:ethertype:ip:tcp]
  Ethernet II, Src: Xerox 80286 (00:00:00:00:00:00), Dst: 52:55:00:00:00:00 (5-
```

Legenda: Figura 3 - Saída detalhada (Verbose) mostrando campos específicos do IP e TCP, equivalente ao painel de detalhes do Wireshark GUI.

3.4. Resumo de Parâmetros Avançados

Parâmetro	Propósito Técnico
-r	Read: Lê pacotes de um arquivo gravado.
-w	Write: Grava pacotes brutos em um arquivo (formato pcap).
-c	Count: Interrompe a operação após N pacotes.
-V	Verbose: Expande detalhes de todas as camadas de rede.
-x	Hex/ASCII: Exibe o dump de memória bruta do pacote.
-q	Quiet: Suprime a saída de pacotes (ideal para gerar estatísticas).

4. Mapeamento MITRE ATT&CK

- **Tática:** TA0007 - Discovery
- **Técnica:** T1046 - Network Service Discovery (Análise de campos específicos do TCP para identificação de SO e serviços).
- **Tática:** TA0010 - Exfiltration
- **Técnica:** T1048 - Exfiltration Over Alternative Protocol (Análise de bytes via -x para detectar dados ocultos em campos de protocolo).

5. Conclusão e Mitigação

O uso de parâmetros avançados do TShark transforma a ferramenta em uma poderosa estação de trabalho forense. A combinação de -V e -x é fundamental para identificar **esteganografia de rede** ou **C2 (Command and Control)** que utilize campos não padronizados. Recomenda-se sempre filtrar o tráfego antes de aplicar o modo verboso para manter a eficiência da análise.

Automação de Captura: Condições de Parada e Buffer Circular no TShark

1. Cenário/Problema

Em ambientes de monitoramento contínuo ou resposta a incidentes, capturar tráfego de rede sem restrições pode esgotar rapidamente o armazenamento em disco. O analista precisa configurar o TShark para realizar capturas inteligentes, definindo limites de tempo, tamanho de arquivo ou operando em um sistema de "buffer circular" (substituindo arquivos antigos) para garantir a coleta de evidências sem interromper a operação do servidor.

2. Ferramentas Utilizadas

- **TShark** (Wireshark CLI).
- **Parâmetros de Autostop (-a)**.
- **Parâmetros de Ring Buffer (-b)**.

3. Evidências Técnicas (Logs/Comandos)

3.1. Condições de Parada Automática (-a)

O parâmetro **-a** (Autostop) define condições para que o TShark encerre a execução automaticamente após atingir um critério específico.

- **Duração:** Para após X segundos.
- **Tamanho:** Para após o arquivo atingir X kilobytes.
- **Arquivos:** Para após gerar X arquivos.

```
# Iniciar sniffing e parar após 2 segundos, limitando arquivos a 5kb e no máximo 5 arquivos
tshark -w autostop-demo.pcap -a duration:2 -a filesize:5 -a files:5
```

3.2. Controle de Buffer Circular (-b)

O parâmetro **-b** (Ring Buffer) é utilizado para capturas de longa duração. Ao contrário do autostop, ele mantém o TShark em execução, mas rotaciona os arquivos. Quando o limite de arquivos (**files:X**) é atingido, o TShark começa a sobrescrever o arquivo mais antigo.

```
# Comece a monitorar o tráfego e pare após 2 segundos, salvando o dump em 5 arquivos, cada um com 5 kb.
tshark -w autostop-demo.pcap -a duration:2 -a filesize:5 -a files:5
```

```

ubuntu@...:~/Desktop/exercise-files$ tshark -w autostop-demo.pcap -
a duration:2 -a filesize:5 -a files:5
Capturing on 'ens5'
13
ubuntu@...:~/Desktop/exercise-files$ ls
autostop-demo_00001_20260302224734.pcap
autostop-demo_00002_20260302224735.pcap
autostop-demo_00003_20260302224735.pcap
autostop-demo_00004_20260302224735.pcap
autostop-demo_00005_20260302224735.pcap
demo.pcapng
write-demo.pcap

```

Legenda: Figura 2 - Listagem de diretório (ls) mostrando a rotação de arquivos gerada pelo buffer circular.

3.3. Comparativo de Parâmetros de Controle

Recurso	Parâmetro -a (Autostop)	Parâmetro -b (Ring Buffer)
Comportamento	Encerra o processo TShark.	Mantém o processo em loop infinito.
Uso de Disco	Limitado pelo critério de parada.	Estabilizado pela rotação de arquivos.
Cenário Ideal	Capturas forenses pontuais.	Monitoramento 24/7 de tráfego.

4. Mapeamento MITRE ATT&CK

- **Tática:** TA0007 - Discovery
- **Técnica:** T1040 - Network Sniffing (Uso de buffers circulares para monitoramento persistente de exfiltração de dados).

5. Conclusão e Mitigação

A implementação de condições de parada e buffers circulares é uma prática recomendada de **Engenharia de Detecção**. Ela previne incidentes de negação de serviço (DoS) local por preenchimento de disco e permite que analistas de SOC mantenham janelas de visibilidade histórica (ex: as últimas 24 horas de tráfego) de forma automatizada. Nota: Estes parâmetros são exclusivos para o modo de captura em tempo real e não podem ser aplicados na leitura de arquivos PCAP pré-existentes.

Estratégias de Filtragem com TShark (Capture vs. Display Filters)

1. Cenário e Fundamentação Teórica

Durante a análise de tráfego em redes de alta densidade, a captura indiscriminada de pacotes pode causar o esgotamento de recursos de hardware e dificultar a identificação de incidentes. Este estudo foca na diferenciação técnica entre as duas camadas de filtragem do TShark: **Filtros de Captura** (eficiência em tempo real) e **Filtros de Exibição** (precisão na análise pós-captura).

2. Metodologia de Filtragem

O aprendizado baseou-se na aplicação de duas sintaxes distintas, cada uma com objetivos específicos dentro do ciclo de vida da análise de pacotes.

2.1. Filtros de Captura (-f)

Os filtros de captura utilizam a sintaxe **BPF (Berkeley Packet Filter)**. Eles são aplicados no kernel do sistema operacional antes que o TShark processe o pacote, sendo a forma mais eficiente de descartar tráfego irrelevante.

- **Objetivo:** Reduzir o volume de dados gravados no disco.
- **Aplicação:** Ideal para isolar protocolos específicos (ex: apenas tráfego de DNS ou HTTP) em redes muito movimentadas.

Exemplo de Sintaxe: ``bash

Comando para capturar apenas tráfego destinado à porta 80 (HTTP)

```
tshark -i ens5 -f "tcp port 80" -w captura_http.pcap
```

•

2.2. Filtros de Exibição (-Y)

Diferente dos filtros de captura, os filtros de exibição utilizam a sintaxe rica e detalhada do Wireshark. Eles não removem dados do arquivo original, mas permitem que o analista "esconda" o ruído para focar em campos específicos de protocolos.

- **Objetivo:** Investigação profunda e busca de Indicadores de Comprometimento (IoCs).
- **Vantagem:** Permite o uso de operadores lógicos complexos e análise de campos internos (como flags TCP ou URIs HTTP).

Exemplo de Sintaxe:

Bash

Comando para analisar um arquivo existente em busca de pacotes SYN de um IP específico

```
tshark -r demo.pcapng -Y "ip.src == 145.254.160.237 && tcp.flags.syn == 1"
```

-

3. Comparativo Técnico de Parâmetros

Recurso	Parâmetro	Sintaxe Utilizada	Momento de Execução
Filtro de Captura	-f	BPF (Sintaxe de baixo nível)	Durante o Sniffing (Tempo Real)
Filtro de Exibição	-Y	Wireshark (Sintaxe de alto nível)	Durante a Leitura (Pós-Captura)

4. Mapeamento de Contexto (MITRE ATT&CK)

- **Tática: Discovery (TA0007)**
- **Técnica: Network Sniffing (T1040):** O conhecimento desses filtros permite que um analista monitore exfiltração de dados de forma cirúrgica, filtrando protocolos de tunelamento ou comunicações de Comando e Controle (C2) ocultas em tráfego legítimo.

5. Conclusão do Aprendizado

A análise técnica demonstra que a maestria no TShark exige a compreensão de quando aplicar cada filtro. O uso do filtro de captura (-f) é uma medida de **preservação de recursos**, enquanto o filtro de exibição (-Y) é a principal ferramenta de **triagem forense**. Esta distinção garante que o analista possa lidar com grandes volumes de dados (Big Data) sem perder a precisão necessária para identificar ameaças sofisticadas.

Entendido! Como agora você **praticou** e utilizou o emulador **Terminator** para validar os filtros em tempo real, a documentação volta ao formato padrão de **Evidências Técnicas**, focando na execução e nos resultados observados.

Aqui está a documentação profissional para o seu GitHub:

Implementação de Filtros de Captura (BPF) com TShark e Terminator

1. Cenário/Problema

Em um ambiente de rede com alto volume de tráfego, capturar todos os pacotes gera arquivos excessivamente grandes e consome processamento desnecessário. O objetivo desta prática foi aplicar **Filtros de Captura (BPF)** para selecionar cirurgicamente o tráfego de interesse (como protocolos específicos ou hosts determinados) antes mesmo da gravação no disco, utilizando o emulador de terminal **Terminator** para validar a captura através da geração de tráfego simultânea.

2. Ferramentas Utilizadas

- **TShark** (Analisador de rede via CLI).
- **Terminator** (Emulador de terminal com suporte a split-screen).
- **cURL & Netcat (nc)** (Geradores de tráfego/ruído de rede).
- **Sintaxe BPF** (Berkeley Packet Filter).

3. Evidências Técnicas (Logs/Comandos)

3.1. Configuração do Ambiente de Teste (Terminator)

Para validar os filtros, utilizei o **Terminator** dividindo a tela em dois terminais operacionais:

1. **Terminal Superior (Monitoramento):** Onde o TShark executa o filtro de captura.
2. **Terminal Inferior (Ação):** Onde os comandos de rede são disparados para gerar pacotes correspondentes ao filtro.

3.2. Filtragem de Host e Protocolo HTTP

Apliquei um filtro de captura focado em um IP específico e validei a recepção dos pacotes através de uma requisição web.

Bash

```
# Terminal 1: Iniciando captura filtrada por host  
tshark -f "host 10.10.10.10"
```

```
# Terminal 2: Gerando tráfego HTTP para o host alvo  
curl -v 10.10.10.10
```

3.3. Filtragem de Portas e Protocolo UDP com Netcat

Neste teste, o objetivo foi capturar apenas tráfego em uma porta arbitrária (4444) utilizando o protocolo UDP.

Bash

```
# Terminal 1: Filtro para porta específica e protocolo  
tshark -f "udp port 4444"
```

```
# Terminal 2: Enviando pacotes UDP via Netcat  
nc -u 10.10.10.10 4444 -vw 5
```

```
ubuntu@ip-10-81-142-21: ~$ tshark -f "udp port 4444"
Capturing on 'ens5'
 1 0.000000000 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1
 2 0.000017310 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1
 3 1.000187628 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1
 4 2.000355526 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1
 5 3.000512355 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1
 6 4.000642273 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1
 7 5.000812242 10.81.142.21 → 10.10.10.10  UDP 43 40976 → 4444 Len=1

ubuntu@ip-10-81-142-21: ~ 80x11
ubuntu@ip-10-81-142-21: ~$ nc -u 10.10.10.10 4444 -vw 5
Connection to 10.10.10.10 4444 port [udp/*] succeeded!
ubuntu@ip-10-81-142-21: ~$
```

Legenda: Figura 2 - Validação de filtro UDP, demonstrando a precisão do parâmetro -f ao ignorar outros tráfegos da rede.

4. Tabela de Referência de Filtros Praticados

Tipo de Filtro	Sintaxe BPF Aplicada (-f)	Ferramenta de Teste
Host Específico	host 10.10.10.10	curl / ping
Rede Inteira	net 10.10.10.0/24	nmap (varredura)
Porta Específica	port 4444	nc (Netcat)
Protocolo Puro	icmp ou udp	ping / nc -u

5. Mapeamento MITRE ATT&CK

- **Tática:** TA0007 - Discovery
- **Técnica:** T1040 - Network Sniffing (Utilizado para monitorar exfiltração de dados em portas não convencionais).

6. Conclusão Técnica

A prática demonstrou que o uso de **Filtros de Captura (-f)** é superior à filtragem de exibição quando o objetivo é performance e economia de armazenamento. A utilização do **Terminator** foi essencial para correlacionar a causa (geração do tráfego) e o efeito (captura estruturada), consolidando o conhecimento sobre como isolar tráfego suspeito em incidentes reais de segurança.

Guia Técnico: Investigação com Filtros de Exibição no TShark

1. Cenário/Problema

Após a captura de tráfego, o analista de segurança frequentemente se depara com arquivos PCAP contendo milhares de pacotes. O desafio é realizar uma triagem eficiente para localizar eventos específicos (como requisições HTTP, consultas DNS ou fluxos de um IP suspeito). Nesta etapa, a prática focou no uso de **Filtros de Exibição (-Y)** para reduzir o ruído visual e extrair estatísticas precisas sem alterar o arquivo original.

2. Ferramentas Utilizadas

- **TShark** (Analisador de pacotes via linha de comando).
- **Arquivo de Captura:** demo.pcapng.
- **Utilitários Linux:** n1 (Number Lines) para contagem de resultados.

3. Evidências Técnicas (Logs/Comandos)

3.1. Filtragem por Endereço IP e Protocolo

Diferente dos filtros de captura, os filtros de exibição utilizam a sintaxe completa do Wireshark, permitindo operadores lógicos como ==, && (AND) e || (OR). Durante a prática, observei que o TShark mantém o número original do pacote na captura, o que é vital para correlação forense.

Bash

```
# Filtrar pacotes que contenham o IP 145.253.2.203 (origem ou destino)
tshark -r demo.pcapng -Y 'ip.addr == 145.253.2.203'
```

3.2. Extração de Estatísticas com n1

Como o TShark não exibe nativamente a contagem total de pacotes filtrados na saída padrão, utilizei o comando n1 para numerar as linhas da saída e obter a contagem exata de ocorrências.

Bash

```
# Filtrar pacotes HTTP e contar o total de resultados encontrados  
tshark -r demo.pcapng -Y 'http' | nl
```

4. Resolução de Desafios (Prática Forense)

Com base na análise do arquivo `demo.pcapng`, foram obtidos os seguintes dados para investigação:

1. **Contagem por IP Específico:**

- **Comando:** `tshark -r demo.pcapng -Y 'ip.addr == 65.208.228.223' | nl`
- **Resultado:** Identificação do volume total de interações com o host.

2. **Contagem por Porta TCP:**

- **Comando:** `tshark -r demo.pcapng -Y 'tcp.port == 3371' | nl`
- **Resultado:** Isolamento de tráfego em portas de serviço específicas.

3. **Filtragem por Origem (Source):**

- **Comando:** `tshark -r demo.pcapng -Y 'ip.src == 145.254.160.237' | nl`
- **Resultado:** Determinação de quantos pacotes foram disparados pelo host investigado.

5. Mapeamento MITRE ATT&CK

- **Tática:** TA0007 - Discovery
- **Técnica:** T1046 - Network Service Discovery (A filtragem de exibição permite identificar portas abertas e serviços ativos em comunicações passadas).

6. Conclusão Técnica

A aplicação de filtros de exibição (`-Y`) é a base da análise forense de rede moderna. A flexibilidade da sintaxe permite que o analista busque por detalhes granulares, como códigos de erro HTTP (`http.response.code == 200`) ou tipos de consulta DNS. A integração com utilitários de texto do Linux (`nl`, `grep`, `wc -l`) potencializa o TShark, transformando-o em uma ferramenta de automação robusta para triagem de incidentes.